

Developer Guide – INFOB318

Individual Project: EatMeSlow

Maria Boubekeur

Bachelor 3 – Computer Science
Academic Year 2024–2025

Developer Guide

EatMeSlow

Maria Boubekeur

May 15, 2025

Contents

1 Introduction **2**

2 General Information **2**

3 Specifications and Development Environment **2**

4 Compilation and Execution **3**

4.1 Launching the Backend 3

4.2 Launching the Frontend 3

4.3 Populating the Database 3

5 Deployment with Docker and Docker Compose **3**

5.1 Manual Build 3

5.2 Combined Deployment with Docker Compose 3

6 Software Architecture **4**

6.1 Frontend (Client Side) 4

6.2 Backend (Server Side) 4

6.3 Data Model 4

7 Runtime Behavior **5**

8 Technical Rationale **5**

9 Future Improvements **5**

10 Code Examples **6**

10.1 FastAPI Route for Logging Recipe Consumption (Backend) 6

10.2 User Authentication (Route `/auth`) 6

1 Introduction

This document is intended for any developer seeking to understand, maintain, or further develop the EatMeSlow project. It provides a comprehensive overview of the system architecture, development environment, utilized technologies, and key technical choices.

2 General Information

- **Project name**: EatMeSlow
- **Purpose**: Web application for nutritional tracking
- **Project directory**: Structured within the `project/` folder
- **Source code**:
  - Frontend: `project/src`
  - Backend: `project/backend`
  - Database script: `project/backend/app/scripts/populate_db.py`
- **User documentation**: Located in `project/docu` as PDF file `userguide.pdf`

3 Specifications and Development Environment

- **Main objective**: To develop a web application that allows users to monitor their food consumption and explore recipes aligned with their nutritional goals.
- **Key expected features**:
  - User account registration and login.
  - Recipe exploration with filtering capabilities (budget, calories, preferences).
  - Addition of selected recipes to daily consumption tracking.
  - Visualization of nutritional history using graphical representations.
- **Technical constraints**:
  - Local deployment using Docker (both frontend and backend).
  - Backend developed in Python using the FastAPI framework.
  - Frontend developed in React.
  - Local SQLite database.
  - Development tools: Visual Studio Code
  - Testing: Manual testing via web browser and FastAPI interactive docs (/docs)
  - Development OS: macOS

- **Target audience**: Adults seeking to independently monitor and improve their dietary habits through a user-friendly interface.

4 Compilation and Execution

4.1 Launching the Backend

```
cd project/backend
pip install -r requirements.txt
uvicorn app.main:app --reload
```

4.2 Launching the Frontend

```
cd project
npm install
npm start
```

4.3 Populating the Database

```
cd project/backend
python -m app.scripts.populate_db
```

The frontend will be available at: `http://localhost:3000`

The backend API documentation can be accessed at: `http://localhost:8000/docs`

5 Deployment with Docker and Docker Compose

5.1 Manual Build

Backend:

```
docker build -t eatmeslow-backend ./project/backend
docker run -p 8000:8000 eatmeslow-backend
```

Frontend:

```
docker build -t eatmeslow-frontend ./project
docker run -p 3000:3000 eatmeslow-frontend
```

5.2 Combined Deployment with Docker Compose

Create a `docker-compose.yml` file at the root of the project:

```
version: "3.8"

services:
  backend:
    build:
```

```
    context: ./project/backend
  ports:
    - "8000:8000"
  restart: always

frontend:
  build:
    context: ./project
  ports:
    - "3000:3000"
  restart: always
  depends_on:
    - backend
```

Then start all services with:

```
docker-compose up --build
```

6 Software Architecture

The project is based on a client-server architecture.

6.1 Frontend (Client Side)

- Developed using React.
- Manages the user interface, routing, and display logic.
- Communicates with the backend through a RESTful API.

6.2 Backend (Server Side)

- Developed using FastAPI.
- Organized into three main layers:
  - API Interface: FastAPI routes (e.g., `/recipe`, `/user`)
  - Validation and filtering logic
  - Interaction with the SQLite database

6.3 Data Model

The project uses SQLAlchemy ORM to define the relational structure of the database:

- **Utilisateur**: Stores user information such as ID, name, email, and hashed password.
- **Aliment**: Represents available foods with nutritional values and cost.
- **Recette**: Represents a combination of foods forming a dish.

- **Recette_Aliment**: Many-to-many association table linking foods and recipes.
- **Recipe_Consommation**: Logs user-consumed recipes along with consumption date.

Each user can consume multiple recipes, and each recipe is composed of multiple food items.

7 Runtime Behavior

Example: Creating a User Account

1. The user fills out a registration form (name, email, password, confirmation).
2. A POST request is sent to the backend endpoint `/register`.
3. FastAPI verifies the input fields, checks for email uniqueness, and ensures password confirmation matches.
4. If all validations pass, a new user is added to the SQLite database.
5. A JSON response is returned to the frontend confirming successful account creation.

Example: Logging a Recipe Consumption

1. The user clicks on "Add to my consumption" from the *Recipes* page.
2. A POST request to `/log-recipe` is sent to the backend with the recipe ID and consumption date.
3. FastAPI validates the inputs, creates a new entry in the `Recette_Consommation` table, and returns a confirmation message.
4. The recorded consumption will be reflected in the user's statistics upon the next query of their history.

8 Technical Rationale

- **FastAPI**: Chosen for its speed, native support for HTTP request handling, and familiarity from prior projects.
- **React**: Selected for its modularity and ability to build dynamic, component-based user interfaces.
- **SQLite**: Used for its simplicity and because it does not require a dedicated server.

9 Future Improvements

- Add a user profile page where users can view and edit their personal information.
- Integrate automated testing using Pytest or other appropriate testing frameworks.

10 Code Examples

This section provides sample backend code to illustrate key application behavior.

10.1 FastAPI Route for Logging Recipe Consumption (Backend)

```
@router.post("/log-recipe")
def log_recipe_consumption(
    consumption: RecipeConsumptionCreate,
    db: Session = Depends(get_db),
    current_user: Utilisateur = Depends(get_current_user)
):
    assert consumption.recipe_id > 0
    new_consumption = Recette_Consommation(
        id_user=current_user.id_user,
        id_recette=consumption.recipe_id,
        date_consommation=consumption.consumption_date
    )
    db.add(new_consumption)
    db.commit()
    return {"message": "Recipe consumption successfully recorded"}
```

10.2 User Authentication (Route /auth)

```
@router.post("/auth")
def login(
    request: Request,
    mail: str = Form(...),
    mot_de_passe: str = Form(...),
    db: Session = Depends(get_db)
):
    assert mail != "", "Email must not be empty"
    assert mot_de_passe != "", "Password must not be empty"

    user = db.query(Utilisateur).filter(Utilisateur.mail == mail).first()

    if not user or not verify_password(mot_de_passe, user.mot_de_passe):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                            detail="Invalid credentials")

    access_token = create_access_token(data={"sub": user.mail})

    return JSONResponse(content={
        "access_token": access_token,
        "token_type": "bearer"
```